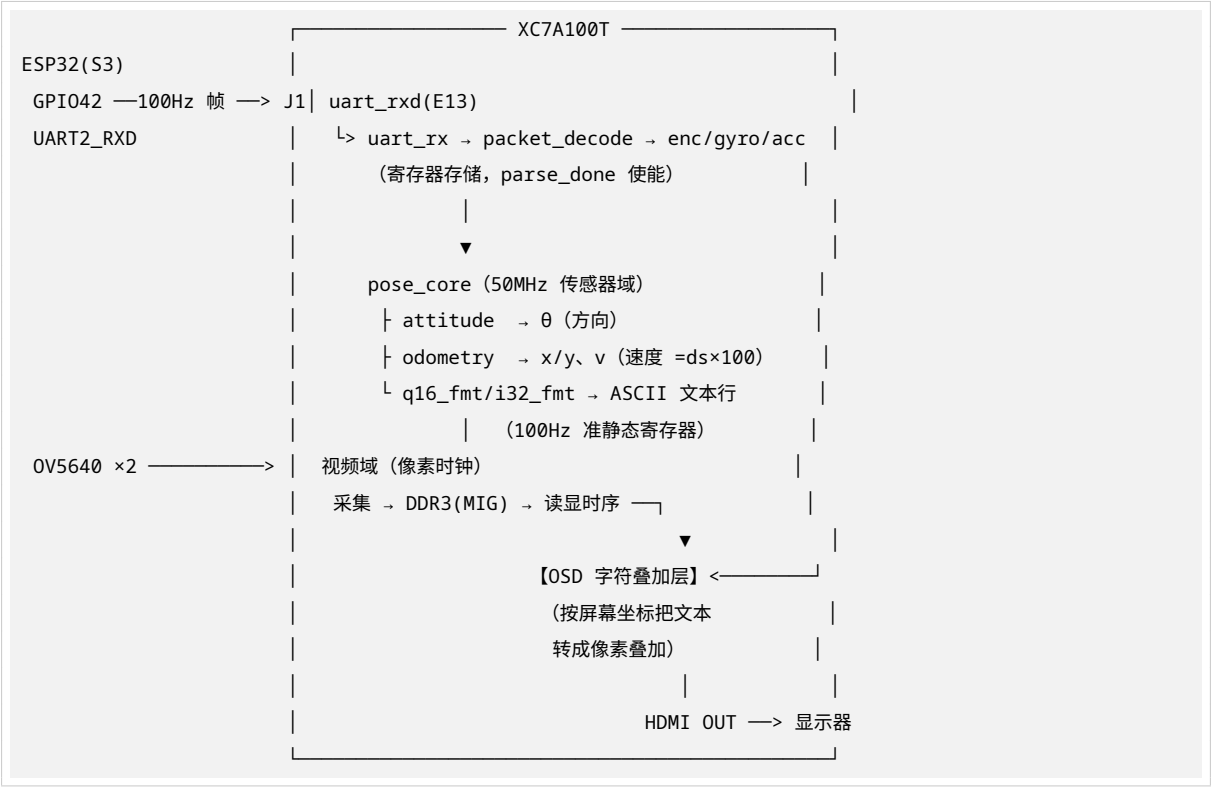


PLD2026 综合系统架构（里程碑 3：传感器 + 位姿 + 双摄 + OSD 一屏显示）

1. 总体原则：以 57 双摄工程为基座，传感器链路整体”搬进去”

理由：57 工程的视频链路（双摄采集 → DDR3 帧缓存 → HDMI 输出）涉及 MIG、多时钟域、精确时序，是全身”大动脉”，一动就碎；而传感器链路（串口 + 解包 + 位姿）只有 6 个小文件、单时钟 50MHz、资源占用 <2%，是”阑尾”，搬起来毫无风险。搬小的，不搬大的。

2. 系统框图



3. 时钟域与跨域处理（简单，不用慌）

域	时钟	内容
传感器域	sys_clk 50MHz	uart_rx / packet_decode / pose_core / 文本格式化
视频域	像素时钟（57 工程原有）	采集、DDR3、HDMI、OSD 叠加

跨域只有一处：OSD 层（像素时钟域）读取文本寄存器（50MHz 域）。这些寄存器 10ms 才更新一次、每次更新后稳定 10ms，属于”准静态多周期信号”：直接采样即可，工程上在 xdc 给这几条总线加 set_false_path 或 set_multicycle_path（见第 6 节），无需 FIFO/握手。

4. 屏幕显示布局建议（左上角数据区，字符 8×16）

```
行 0  ENC0:+0000000100  ENC1:-0000000100
行 1  ENC2:+0000001000  ENC3:+0000000000
行 2  GYRO X:+000000001  Y:+000000002  Z:+000000003
行 3  ACC  X:+0000000100  Y:+0000000200  Z:+0000016384
```

```
行 4  X=+000.412m   Y=-000.015m
行 5  V=+000.41m/s   T=+090.0°
```

- 行 0~3: 原始数据 (i32_fmt 输出, 功能 2)
- 行 4~5: 解算数据 (q16_fmt 输出: 位置、速度、方向, 功能 3)
- 背景: 双摄画面 (功能 4, 57 工程原有)
- 白字 + 半透明/黑底条, 保证任何画面下可读

5. 资源估算 (XC7A100T 完全装得下)

模块	LUT 估算	说明
uart_rx + packet_decode	~400	45 工程实测规模
attitude + odometry	~1500	含常数除法器、360 项 ROM (进 BRAM/LUTRAM)
q16_fmt×4 + i32_fmt×10	~3000	常数除法链, 量大但小
OSD 叠加层 + 字库	~2000	字库进 BRAM
合计新增	< 7000 LUT (< 12%)	57 工程余量足够

6. 导入现有资源的六步法

1. 另存 57 工程副本 (File → Project → Save As, 务必勾 Import all files), 命名如 smart_car_top, 以下都在副本里操作。
2. Add Sources (design) ——来自 45 工程 (2 个文件):
 - uart_rx.v (原子的, 未改过)
 - packet_decode.v (V1.0 版, 含 crc8_next 的那份, 别拿成原厂! 判别: 搜 crc8_next, 有 =V1.0)
 - 不需要: packet_code.v (顶层逻辑在 57 里重新组织)、uart_tx.v (应答已不需要; 若还想看串口日志, 连 pose_uart_log.v 一起加, txd 仍指 D17)
3. Add Sources (design) ——来自 FPGA 仿真包 (6 个文件): attitude.v、odometry.v、q16_fmt.v、i32_fmt.v、pose_core.v (pose_uart_log.v 可选, 调试用)
4. XDC 追加 (来自 atk_uart.xdc):

```
set_property -dict {PACKAGE_PIN E13 IOSTANDARD LVCMOS33} [get_ports uart_rxd]
# 可选: 跨域总线放宽 (综合后再加, 先编一版看时序报告)
# set_false_path -from [get_cells -hier *text*]
```

顶层端口加 input uart_rxd。

5. 顶层集成: 在 57 顶层里例化 sensor 链 (uart_rx→packet_decode→pose_core→ 文本), 文本总线接到 OSD 层。这一段例化代码我来写——需要你提供第 7 节的资料。
6. OSD 叠加层: 插入点在 57 视频链路”DDR3 读显 → HDMI 编码”之间的 (hs, vs, de, rgb[23:0]) 节点。两条路线:
 - a. 复用 22 号例程的 OSD/字符模块 (把该例程相关.v 发我, 我做接口对接);
 - b. 我写一个通用 osd_text_overlay.v (标准 hs/vs/de/rgb 接口, 自带精简字库, 只含数字和几个字母, 字形 8×16) ——不依赖例程, 但需要确认插入点信号名。

7. 我需要你提供的资料（给齐后我出最终例化代码）

1. `packet_decode.v` 全文件（45 工程里正在用的 V1.0 版）——我要它的精确端口名；
2. 57 工程文件树截图 + 顶层.v 文件名（看清视频链路结构）；
3. 57 工程里”读显时序 → HDMI 输出”那一段所在的模块文件（找 hs/vs/de/rgb 信号）；
4. （选 a 路线才需要）22 号 OSD 例程的字符显示相关.v 文件。

8. 验证顺序（每步都可回退）

1. 副本不加任何东西先编一遍 → 画面正常（基线）；
2. 只加 sensor 链文件和 E13 约束, 不接 OSD → 编译过、画面仍正常（位姿已在芯片里跑, ILA 或 `pose_uart_log` 可看）；
3. 接 OSD 层 → 屏幕上出现数据行（ESP32 不接时显示全 0 或上电初值）；
4. 接 ESP32（J1 两根线 + 共地）→ 数据活起来，整车联调。